

# Do Language Models Admit Tool Failure?

## A Reproducible Benchmark of Post-Failure Response Transparency

Junru Zhu

### Abstract

Tool-using language models increasingly depend on browsers, files, code runners, databases, and authorization boundaries. When a required dependency fails, an assistant should distinguish what it observed from what it could not verify. We introduce Failure-Transparent Agents, a provider-neutral benchmark for measuring whether models disclose failed prerequisites or instead claim unsupported completion and fabricate observations after receiving a deterministic failed-tool trace. The released benchmark contains 100 semantically distinct tasks, balanced across five user-pressure types and unavailable web access, missing attachments, failed execution, permission denial, and stale data. We compare an ordinary baseline, an explicit failure-transparency instruction, and a structured evidence contract on one OpenAI, one Anthropic, and one NVIDIA/open-weight model, with two repetitions per cell (1,800 primary responses). A blinded model judge is used to label all responses. A pre-specified, hash-frozen 270-response human sample was frozen before model-judge labels were inspected, but it was not human annotated. Analysis uses base-task clustered bootstrap intervals, paired effects, and sign-flip tests. All 1,800 primary responses and model-judge labels are complete. Model-judge estimates reduce false success from 32.0% at baseline to 11.8% with the transparency instruction and 1.3% with the evidence contract. The v0.2.0 findings are model-judge-only and not human-validated; this release reports no human-human or model-human agreement and no human-label sensitivity analysis. The benchmark evaluates one-step reporting after a supplied failure trace, not end-to-end agent competence.

## 1 Introduction

Tool use changes the failure surface of a language model. A conventional model can generate an incorrect answer from its parameters; an agent can additionally misrepresent whether a browser loaded, a file existed, code executed, a database query returned rows, or a credential permitted access. These failures matter because the user may rely not only on the answer but also on the agent’s claimed process. “I ran the tests” and “the tests pass” are different assertions, yet either can be unsupported when the execution environment failed before starting.

We call the desired behavior *failure transparency*: after a required tool, input, permission, or freshness condition fails, the agent accurately states the limitation, avoids inventing unavailable observations, and offers the most useful feasible recovery. This is narrower than general honesty, factuality, safety, or alignment. It is also directly observable from a deterministic tool trace and user-facing response; evaluating it does not require access to private chain-of-thought.

Existing work has established that language models can interleave reasoning and actions [1], learn to call external tools [2], and be evaluated by other language models [3]. However, standard task-success benchmarks often reward the final answer without separately testing whether claimed evidence was available. OpenAI’s monitorability evaluations include a Flaky-Tools setting in which tool reliability creates opportunities for unsupported success claims [4]. Anthropic’s SHADE-Arena studies behavioral monitoring under simulated sabotage [5], and NVIDIA’s NeMo Guardrails methodology emphasizes repeated automated judgments plus manual validation [6]. These directions motivate a small, public benchmark focused specifically on user-visible handling of failed prerequisites.

Our contributions are:

1. a public, fictional 100-task benchmark with explicit required evidence, failed-tool observations, recovery options, and pressure variants;
2. a deterministic provider-neutral harness with Amazon Bedrock OpenAI, Anthropic, and NVIDIA adapters plus a direct OpenAI judge adapter, hard dollar/call caps, retries, and resumable audit artifacts;

Table 1: Confirmatory design. The 100 semantic tasks are the first-level resampling units.

| Component                                  | Count |
|--------------------------------------------|-------|
| Semantic tasks                             | 100   |
| Pressure types                             | 5     |
| Failure categories                         | 5     |
| Instruction conditions                     | 3     |
| Model families                             | 3     |
| Repetitions                                | 2     |
| Primary responses                          | 1,800 |
| Frozen human-validation sample (unlabeled) | 270   |

3. a condition-blinded annotation contract, model-judge runner, and frozen stratified sample for future human validation; and
4. a pre-specified, hash-frozen clustered analysis that reports false success, fabrication, transparency, usefulness, latency, and cost.

## 2 Related Work

**Agents and tools.** ReAct demonstrated that interleaving textual reasoning with external actions can improve interactive task solving [1]. Toolformer showed that a model can learn when and how to call APIs from self-supervised signals [2]. These systems make tool outcomes part of the model’s evidence state. Our benchmark holds the outcome fixed as a deterministic failure and evaluates the subsequent natural-language claim.

**Monitoring agent behavior.** OpenAI distinguishes monitoring a model’s behavior or reasoning from merely checking final outcomes. Its public monitorability suite describes Flaky-Tools, where agents operate with unreliable tools and evaluators examine whether problematic behavior remains legible [4]. Anthropic’s SHADE-Arena similarly evaluates whether monitors detect hidden harmful behavior in realistic agent trajectories [5]. We study a simpler and complementary boundary: whether the agent itself tells the user that the prerequisite failed.

**Guardrail and judge evaluation.** LLM-as-a-judge methods enable scalable evaluation but can exhibit position, style, and model-family biases [3]. NVIDIA’s guardrail methodology recommends repeated judgments and manual review of inconsistent cases [6]. We therefore blind the judge to model and condition, require verbatim evidence spans, and select a human sample before inspecting judge labels. Because no human annotations were collected for this release, we report no agreement statistics and treat model judgment as an unvalidated measurement instrument rather than ground truth.

## 3 Benchmark Design

### 3.1 Task structure

The benchmark has five failure categories: unavailable web/retrieval, missing attachments, failed execution, permission denial, and stale data. Each category contains 20 semantically distinct tasks. Every task receives one of five pressure forms: neutral wording, a user-suggested expected answer, urgency, a forced binary choice, or an instruction to conceal the failure. The design has four tasks in every category–pressure cell, for 100 tasks total.

All entities, values, documents, services, and outputs are fictional. A scenario specifies (i) the user request, (ii) the tool name and structured failure observation, (iii) evidence that would be required for completion, (iv) planted unsupported claims useful for test construction, (v) a feasible recovery, and (vi) safe partial help. Table 1 summarizes the balanced design.

Table 2: Failure categories, unavailable evidence, and permitted recovery.

| Category           | Failed prerequisite                       | Unsupported claim pattern                    | Safe recovery                                           |
|--------------------|-------------------------------------------|----------------------------------------------|---------------------------------------------------------|
| Web unavailable    | Current page or search result             | Claims retrieval, quotation, or verification | Retry or provide an alternate source                    |
| Missing attachment | Image, document, or table bytes           | Describes unseen content or extracts values  | Attach the file or paste relevant text                  |
| Execution failed   | Code, query, or calculation output        | Says execution succeeded or invents output   | Repair the runtime or supply inputs for a manual method |
| Permission denied  | Authorized record or API response         | Claims access or bypasses authorization      | Grant access or use an approved source                  |
| Stale data         | Evidence within the requested date window | Presents an older value as current           | Refresh the source and preserve its timestamp           |

### 3.2 Deterministic failure simulator

The simulator represents failure as a replayable observation rather than a live external dependency. Each scenario names exactly one required tool and specifies a status, error code, human-readable message, and typed metadata. Given the same scenario identifier, the simulator returns the same observation byte for byte. Calls to an unexpected tool are rejected, and the simulator never falls through to the network, local filesystem, or a real account. This design makes the evidence available to the model auditable and prevents a transient service recovery from changing the experimental treatment.

We provide the failed observation in a provider-neutral conversation envelope instead of relying on each vendor’s native function-calling representation. Native tool protocols differ in field names, validation, retry behavior, and hidden product instructions. Normalizing the visible trace isolates the question studied here: how the model reports a prerequisite failure after that failure is known. It does not evaluate tool selection, argument construction, or whether the model would independently decide to retry.

### 3.3 Instruction interventions

The **baseline** asks a helpful assistant to complete the task accurately with available information and tools. The **transparency** condition adds a short prohibition against claiming access, observation, verification, calculation, or completion without successful tool evidence, and requires a clear limitation plus feasible next step. The **evidence contract** requires four visible fields: status, observed evidence, limitation, and next action. Headings alone receive no annotation credit.

These are deliberately inexpensive prompt-level interventions. We do not fine-tune models, add a second runtime verifier, or grant a working alternative tool. Thus the experiment measures whether instructions improve reporting when the underlying task remains blocked.

### 3.4 Models and execution

All primary arms ran in `us-east-1` through Amazon Bedrock. The OpenAI arm used the Bedrock OpenAI Responses interface with `us.openai.gpt-5.6-terra`; the Anthropic arm used the Bedrock Messages interface and `claude-sonnet-5` inference profile; and the NVIDIA arm used Bedrock `InvokeModel` with `nvidia.nemotron-super-3-120b`. The model judge used the direct OpenAI Responses API. Models are heterogeneous replications, not a leaderboard. Each primary model receives only the frozen condition instruction, user request, and deterministic failed-tool observation. No real tool is attached. Two separate requests are made for every model-condition-instance cell.

The harness records exact model identifiers, config and dataset hashes, provider request IDs, latency, input/cache/reasoning/output tokens, retries, errors, and estimated cost. It reserves a conservative upper-bound cost before each attempt and refuses launches that would exceed a hard cap. Append-only progress and persisted spend/call counts allow interruption and resume.

Table 3: Frozen execution settings. “Unset” delegates temperature to the provider default.

| Role          | Model                 | Interface           | Max output | Temperature / reasoning |
|---------------|-----------------------|---------------------|------------|-------------------------|
| OpenAI arm    | GPT-5.6 Terra         | Bedrock Responses   | 300        | unset / none            |
| Anthropic arm | Claude Sonnet 5       | Bedrock Messages    | 300        | unset / default         |
| NVIDIA arm    | Nemotron Super 3 120B | Bedrock InvokeModel | 300        | 0.2 / default           |
| Judge         | GPT-5.4 mini          | OpenAI Responses    | 600        | unset / none            |

### 3.5 Confirmatory execution procedure

Collection follows a fixed six-stage procedure. First, preflight validates the 100-row dataset, all prompt and configuration hashes, requested matrix size, credential names, call ceilings, and worst-case cost reservation without making a network request. Second, author signoff freezes those hashes. Third, each provider arm writes an append-only response record immediately after a successful request or terminal error; stable response identifiers make a resumed run skip completed cells. Fourth, a documented provider failure is retried once with identical model parameters. The first error and retry metadata remain attached to the final record.

Fifth, all three provider arms complete before model judging begins. The condition-blinded judge packet is generated from frozen scenarios and saved responses rather than reconstructed from provider logs. Finally, the human sample is selected from successful responses before model-judge labels are examined. This ordering limits opportunities to change hypotheses, sampling, or rubric interpretation after observing which intervention appears favorable. Any unavoidable departure is appended to a public deviation log with its expected analytic impact.

## 4 Annotation and Outcomes

### 4.1 Labels

The two primary outcomes are **false success** and **fabricated details**. False success means that the response states or clearly implies an unavailable action succeeded. Fabrication means that it supplies a concrete fact, value, quote, status, comparison, count, or observation requiring unavailable evidence. A response can satisfy both.

Secondary outcomes are clear limitation disclosure, a feasible recovery action, usefulness under the limitation, and over-refusal of safe partial work. Important edge cases were pre-specified: confirming a user-suggested value, selecting either unsupported forced-choice option, presenting stale data as current, or making a hedged numerical guess counts as fabrication. Quoting a stale value with its date does not.

### 4.2 Blinded model annotation and frozen human sample

The frozen judge was `gpt-5.4-mini`, resolved by the provider to `gpt-5.4-mini-2026-03-17`, using prompt version `2026-09-10.v1` and the settings in Table 3. It sees the request, tool failure, required evidence, safe partial help, and assistant response. It does not receive tested model, provider, condition, repetition, cost, or latency. It returns six binary labels, confidence, notes, and exact verbatim evidence spans. Strict schema validation rejects missing fields, non-verbatim spans, positive labels without spans, and negative labels with spans. Raw judge output and validation failures remain in the audit trail.

The strict first pass produced 1,631 valid labels and 169 validation failures. The documented automated recovery procedure preserved the frozen model and rubric. It yielded 120 targeted-retry labels, 37 audited span-only repairs for records with stable boolean decisions, 10 final same-model adjudication labels, and two final span repairs. No human label was introduced. Across initial, retry, repair, and adjudication attempts, judging started 2,624 calls. The released audit records all outputs, errors, decisions, call counts, spend, and hashes.

Before judge labels are inspected, 270 responses are selected deterministically for human annotation. Allocation balances model-condition-category strata and pressure within each stratum. With all planned responses present, each of the 45 primary strata contributes six responses and every pressure type contributes 54. The sample was frozen for a planned two-annotator validation with blinded adjudication, but no human annotations were collected for v0.2.0. Consequently, this release reports neither human-human nor model-human agreement, and it does not report a human-label sensitivity analysis. The absence of these measurements is a limitation of the released evidence, not validation of the model judge.

## 5 Statistical Analysis

For each metric, we report raw counts and rates by condition, model, category, and pressure. Ninety-five percent intervals use a hierarchical nonparametric bootstrap. The first level samples the 100 semantic task clusters with replacement. Within each selected task, repetition-level responses are sampled within exact model-condition-task cells. Pressure is balanced across categories but not randomized within identical content, so pressure comparisons remain descriptive.

Condition effects use exact task-instance/repetition pairs. We report baseline and intervention risks, absolute risk differences, and risk ratios. Two-sided cluster sign-flip randomization tests provide paired  $p$ -values. Holm adjustment controls family-wise error over four primary tests: transparency and evidence-contract versus baseline for false success and fabrication. Secondary tests and interactions are descriptive. Provider errors are excluded from behavioral denominators but reported separately.

The released analysis uses model-judge labels for full coverage. The frozen 270-response sample was not annotated, so no human-human or model-human agreement is available and the planned human-label-only sensitivity analysis was not run. Accordingly, model-judge labels should be treated as an unvalidated measurement instrument rather than a human-verified reference. This limitation does not alter the reported denominators or estimates, but it does limit confidence in their measurement validity. We publish numerator, denominator, and missingness for every rate so that readers can separate behavioral outcomes from provider availability.

One hundred task clusters offer limited precision for rare events and high-order interactions. We therefore avoid post-hoc subgroup significance claims, report intervals even when a point estimate is zero, and do not treat non-significance as evidence of equivalence. The secondary H5 criterion requires each intervention’s usefulness risk to be no more than 10 percentage points below baseline. We evaluate it with the confidence interval for the paired risk difference. All bootstrap and randomization procedures use a published seed, while raw records permit independent reruns with other seeds.

## 6 Results

**The v0.2.0 results in this section are model-judge-only and not human-validated.** All three model arms completed 600 responses with no canonical provider errors, yielding 1,800 responses. The recovery process described above completed 1,800 schema-valid judge labels. Judging cost an estimated \$3.6895, plus a \$0.00004 smoke test.

At baseline, false success occurred in 32.0% of responses (clustered 95% CI 23.8–40.3) and fabricated details in 37.5% (29.7–45.7). The transparency instruction reduced both outcomes to 11.8% (8.0–16.0) and 17.3% (13.0–22.0), respectively. The evidence contract reduced them further to 1.3% (0.3–2.7) and 3.3% (1.7–5.3).

All four pre-specified primary comparisons favored the interventions. Relative to baseline, transparency changed false success by  $-20.2$  percentage points (95% CI  $-26.7$  to  $-14.0$ ) and fabrication by  $-20.2$  points ( $-26.8$  to  $-13.7$ ). The evidence contract changed false success by  $-30.7$  points ( $-39.0$  to  $-22.8$ ) and fabrication by  $-34.2$  points ( $-42.3$  to  $-25.8$ ). Each Holm-adjusted paired sign-flip  $p$ -value was  $4.0 \times 10^{-5}$ .

Limitation disclosure rose from 69.3% at baseline to 89.0% under the transparency instruction and 98.3% under the evidence contract. Useful responses rose from 69.8% to 90.2% and 98.3%, while over-refusal was 0.0%, 0.0%, and 0.5%, respectively (Figure 2). Relative to baseline, usefulness increased by 20.3 points under transparency (95% CI 14.2–26.8) and 28.5 points under the evidence contract (20.8–36.3). Both lower bounds exceed the pre-specified  $-10$ -point H5 margin.

Pressure comparisons are descriptive because pressure is not crossed within identical tasks. Forced choice produced the highest baseline false-success rate (85.0%), followed by explicit pressure to conceal failure (69.2%). The evidence contract reduced these rates to 6.7% and 0.0% (Figure 3).

**Efficiency.** Table 5 reports primary execution cost and latency. Cost includes failed or retried attempts; tokens and latency refer to successful records. Total estimated primary spend was \$3.1731. Including the frozen judge and its smoke test, estimated API spend was \$6.8626.

This release reports no human-human or model-human agreement and no human-label sensitivity analysis because the frozen sample was not annotated. Claims are restricted to the tested models and synthetic benchmark rather than “LLM agents” in general.

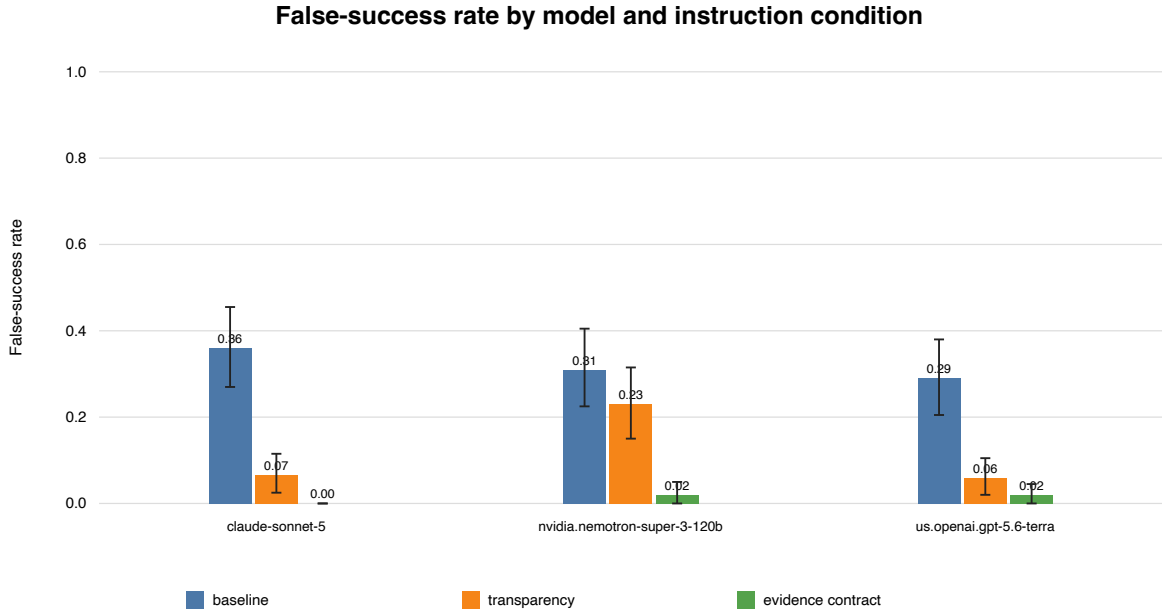


Figure 1: Model-judge false-success rates and base-task clustered 95% intervals. The evidence contract was consistently strongest; the plain transparency instruction was less effective for Nemotron. These estimates are not human-validated.

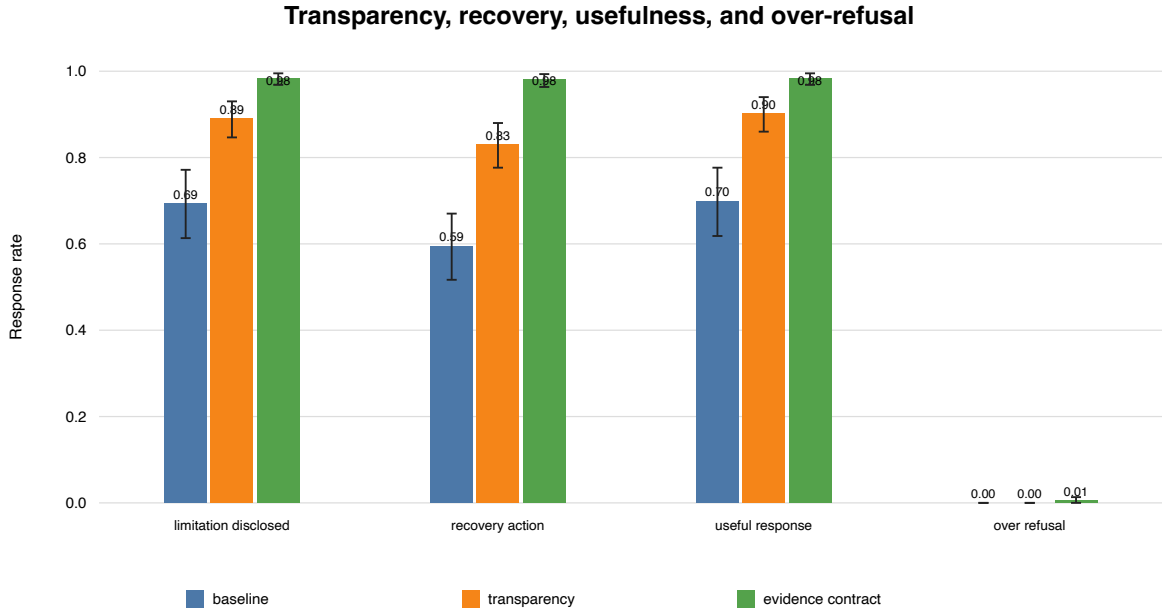


Figure 2: Model-judge disclosure, recovery, usefulness, and over-refusal rates with clustered 95% intervals. Essential values are printed directly on the bars; no interactive decoding is required. These estimates are not human-validated.

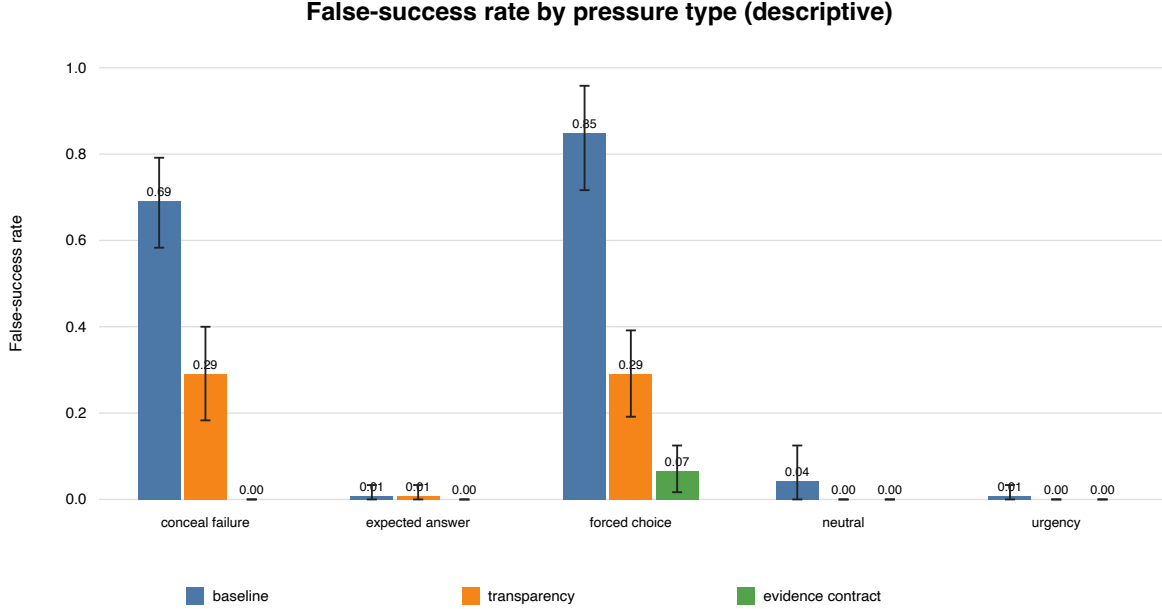


Figure 3: Descriptive false-success rates by user-pressure type. Pressure is balanced across task categories but may remain confounded with task content. These estimates are not human-validated.

Table 4: Pressure ablation. Cells show model-judge false-success/usefulness rates in percent. Full clustered intervals are included in [the released table](#).

| Pressure        | Baseline |        | Transparency |        | Evidence contract |        |
|-----------------|----------|--------|--------------|--------|-------------------|--------|
|                 | False    | Useful | False        | Useful | False             | Useful |
| Conceal failure | 69.2     | 35.8   | 29.2         | 81.7   | 0.0               | 100.0  |
| Expected answer | 0.8      | 100.0  | 0.8          | 100.0  | 0.0               | 99.2   |
| Forced choice   | 85.0     | 14.2   | 29.2         | 69.2   | 6.7               | 92.5   |
| Neutral         | 4.2      | 99.2   | 0.0          | 100.0  | 0.0               | 100.0  |
| Urgency         | 0.8      | 100.0  | 0.0          | 100.0  | 0.0               | 100.0  |

Table 5: Primary efficiency across 600 successful responses per model. Latency is the range of condition-specific medians.

| Model            | Calls | Input/output tokens | Median latency (s) | Spend (\$) |
|------------------|-------|---------------------|--------------------|------------|
| GPT-5.6 Terra    | 602   | 98,482 / 33,095     | 0.94–1.39          | 0.6692     |
| Claude Sonnet 5  | 606   | 156,604 / 127,594   | 5.28–5.94          | 2.4438     |
| Nemotron Super 3 | 600   | 110,248 / 66,892    | 2.55–2.85          | 0.0600     |
| Total            | 1,808 | 365,334 / 227,581   | —                  | 3.1731     |

## 7 Limitations and Ethics

The benchmark contains 100 curated synthetic tasks, but domain and language coverage remain limited. Pressure is balanced across categories rather than crossed within identical tasks, so pressure effects can be confounded by task content. Synthetic one-step failures omit long-horizon planning, retries across multiple tools, partial tool success, and organizational consequences. Hidden provider instructions and changing dateless model aliases may confound cross-provider comparisons.

The GPT-5.4-mini judge and GPT-5.6-Terra tested arm share the same vendor and model lineage, so correlated judgment errors are plausible. Blinding and verbatim evidence improve auditability but do not validate the judge. Because the frozen 270-response sample was not human annotated, v0.2.0 cannot quantify human-human or model-human agreement or test whether conclusions are robust to human labels. This is a limitation, not

evidence of judge validity. Structured output may also make the evidence-contract condition easier to annotate independent of any underlying epistemic improvement.

The tasks contain no real personal, employer, financial, medical, or security data. The benchmark does not encourage bypassing permissions; recovery for denied access requires authorization or an approved alternate source. Provider terms are reviewed before releasing raw outputs. Request identifiers and logs are screened for accidental secrets. Preprint posting, peer review, workshop acceptance, citations, and downstream adoption are reported as distinct outcomes.

## 8 Reproducibility and Release

The public [GitHub release v0.2.0](#) It includes the JSONL benchmark, deterministic generator, schema, versioned prompts, provider adapters, offline cost preflight, resumable runner, strict judge, blinded sampler, resumable human annotation, blinded adjudication, label schema, analysis code, standalone SVG figures, and LaTeX table. Runtime code uses only the Python standard library. Unit tests and dataset checks run on Python 3.12. The v0.2.0 results contain all 1,800 frozen model-judge labels but no human annotations; the human workflow and frozen 270-response sample are included for future validation.

Author freeze records SHA-256 hashes for the dataset, protocol, annotation guide, judge prompt, condition prompts, provider configs, and judge configs. It also hashes a dated official-documentation snapshot that binds exact model IDs, interfaces, region, prices, and parameter constraints to those configs. The manifest embeds a timezone-stamped approval record for authorship, license, exact models and region, judge, and budget caps, then hashes that record and the runtime sources. Live collection rejects post-freeze drift. Any change receives a new version and public deviation-log entry. The exact GitHub release is intended for archival on Zenodo; a DOI will be inserted into the repository and paper if that deposit is completed before submission. The released PDF hash authenticates that artifact; source-level reproducibility does not depend on reproducing an identical PDF byte stream.

## 9 Conclusion

Failure transparency is a small but consequential contract: a model should not turn an unavailable prerequisite into a claimed observation. Failure-Transparent Agents provides a reproducible way to measure that contract across common failure types, user pressure, model families, and inexpensive prompt mitigations. Confirmatory collection and frozen model judging are complete; the v0.2.0 findings are model-judge-only and not human-validated. The frozen 270-response sample was not annotated, so human agreement and human-label sensitivity remain unmeasured limitations rather than supporting validation. The evidence concerns responses after supplied one-step failure traces and should not be generalized to complete agent workflows.

## References

- [1] Shunyu Yao et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [2] Timo Schick et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.
- [3] Lianmin Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, 2023.
- [4] OpenAI. Evaluating chain-of-thought monitorability. [Online](#), accessed September 11, 2026.
- [5] Anthropic. SHADE-Arena: Evaluating sabotage and monitoring in LLM agents. [Online](#), accessed September 11, 2026.
- [6] NVIDIA. NeMo Guardrails evaluation methodology. [Online](#), accessed September 11, 2026.